

## 04. 併發模型 (Concurrency)

### 1. GMP 排程模型

Go 的 Goroutine 如此輕量且高效，歸功於其 runtime 內部的 M:N 排程器 (GMP 模型)。

```
flowchart TD
    subgraph "OS 層級 (Kernel Space)"
        M1["M1 (OS Thread)"]
        M2["M2 (OS Thread)"]
    end

    subgraph "Go Runtime (User Space)"
        P1["P1 (Processor)\n擁有 Local Queue"]
        P2["P2 (Processor)\n擁有 Local Queue"]

        G1(("G"))
        G2(("G"))
        G3(("G"))
        G4(("G"))

        P1 --> M1
        P2 --> M2

        G1 --> P1
        G2 --> P1
        G3 --> P2
    end

    GlobalQueue["Global Queue"] -. "P 的 Local Queue 空了\n會來這裡拿 (或去偷其他 P 的)" .-> P1
    GlobalQueue -. -> P2
    G4 --> GlobalQueue
```

名詞解釋：G (Goroutine), M (Machine/Thread), P (Processor/資源調度者)

### 2. Channel 通訊模式

Go 的併發名言："不要透過共享記憶體來通訊；要透過通訊來共享記憶體"。

```
sequenceDiagram
    participant S as Sender (Goroutine)
    participant C as Channel (Unbuffered)
    participant R as Receiver (Goroutine)

    S->>C: ch <- data (阻塞等待)
    Note over S: Sender 卡住，直到有人接手
    R->>C: <- ch (讀取資料)
    C-->>R: data
    C-->>S: 傳送成功 (解除阻塞)
```

### 3. 多路複用：select

select 讓單一 Goroutine 可以同時等待多個 Channel 的讀寫事件。如果多個 case 同時就緒，它會隨機選擇一個執行 (避免飢餓現象)。

```

flowchart LR
    G["Goroutine\n(select)"]
    CH1["Channel 1\n(Ticker: 每秒觸發)"]
    CH2["Channel 2\n(Data: 收到新訊息)"]
    CH3["Channel 3\n(Quit: 關閉信號)"]

    CH1 -->|case <-ticker.C| G
    CH2 -->|case msg <-dataCh| G
    CH3 -->|case <-quitCh| G

    G -->|"若全無資料"| Default["執行 default (若有)\n否則 Block"]

```

## 4. Context 樹狀連鎖取消

Context 是 Go 用來控制跨 Goroutine 超時與取消的標準方式。當父節點被取消，所有的子孫節點都會收到取消信號。

```

graph TD
    Root["context.Background()"]
    Req["Request Context (User ID: 123)"]
    DB["DB Query Context (Timeout: 5s)"]
    API1["API Call 1 (Timeout: 2s)"]
    API2["API Call 2 (Timeout: 2s)"]

    Root --> Req
    Req --> DB
    Req --> API1
    Req --> API2

    style API1 fill:#f9cfcf,stroke:#ff5555
    Note right of API1: 若 API 1 提早發生 Timeout
    Note left of Req: 呼叫 cancel()
    Req -. "連鎖發送 Done 信號" .-> DB
    Req -. "連鎖發送 Done 信號" .-> API2

```

## 5. Fan-out / Fan-in 模式

經典的高效能併發處理模式：由一個來源分發給多個 Worker 處理 (Fan-out)，再由一個收集器將多個 Worker 的結果合併 (Fan-in)。

```

flowchart LR
    Source["Task Source"]
    W1["Worker 1"]
    W2["Worker 2"]
    W3["Worker 3"]
    Merge["Merge (Fan-in)"]
    Result["Final Result"]

    Source -- "Fan-out (Task Channel)" --> W1
    Source -- "Fan-out" --> W2
    Source -- "Fan-out" --> W3

    W1 -- "Result Channel 1" --> Merge
    W2 -- "Result Channel 2" --> Merge
    W3 -- "Result Channel 3" --> Merge

    Merge --> Result

```